NEOBRIM
Elevating Homes

Software Requirements Specifications (SRS) for TV Class

Requirement #3: TV Class Module

- **Assignee:** Dhanushkumar R
- **Role:** Deep Learning (AI) Intern
- **Date of Assignment:** March 6, 2024
- **Deadline:** March 9, 2024

Software Requirements Specification (SRS) for Home Automation Class

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed specification for the Home Automation class, a pivotal component of the Home Automation System. This class orchestrates the management of multiple appliances across various locations within a home, incorporating advanced features to enhance user comfort and lifestyle.

1.2 Scope

The Home Automation class is designed to handle a diverse set of appliances, such as lights, thermostats, fans, and TVs, considering different locations within a home. The system aims to achieve intelligent decision-making through interaction with the Proximal Policy Optimization (PPO) class, providing an adaptable and user-centric home automation experience.

2. System Overview

2.1 Home Automation Class

The Home Automation class acts as the brain of the home automation system, responsible for managing variables, controlling appliances, and utilizing reinforcement learning for decision-making.

2.2 Key Features

1. **Variable Management:**
    - Manage dynamic variables related to appliance states, user preferences, environmental conditions, and locations.
2. **Appliance Control:**
    - Control a comprehensive range of appliances, considering user preferences and learned patterns.
3. **Decision-Making Interface:**
    - Interact with the Proximal Policy Optimization (PPO) class for intelligent, adaptive decision-making.

4. **Advanced Settings:**
    - Implement advanced settings for appliances, including color control for lights and channel selection for TVs.

3. Functional Requirements

3.1 Variables

3.1.1 Appliance States

1. **Lights States (Type: Dictionary):**
    - Represents the on/off state and color settings of lights at different locations.
2. **Thermostats States (Type: Dictionary):**
    - Indicates the on/off state and temperature settings of thermostats at different locations.
3. **Fans States (Type: Dictionary):**
    - Denotes the speed levels or on/off state of fans at different locations.
4. **TVs States (Type: Dictionary):**
    - Represents the on/off state, current channel, and volume settings of TVs at different locations.

3.1.2 User Preferences

1. **Lighting Preferences (Type: Dictionary):**
    - Stores user-defined lighting preferences, including brightness, color, and transition effects for different locations.
2. **Temperature Preferences (Type: Dictionary):**
    - Holds user-set temperature preferences for different periods and locations.
3. **Fan Speed Preferences (Type: Dictionary):**
    - Stores user-set preferences for fan speed at different locations.

3.1.3 Environmental Conditions

1. **Temperature (Type: Float):**
    - Current ambient temperature in degrees Celsius.
2. **Humidity (Type: Float):**
    - Current humidity level in percentage.

3.1.4 Location Information

1. **Location (Type: String):**

- Represents the current location within the home environment.

3.2 Functions

3.2.1 Variable Management

1. **fetch_appliance_states(self, location: str) -> dict:**
   - Retrieves the current states and settings of all appliances at a specific location.
2. **update_lighting_preferences(self, location: str, brightness: int, color: str, transition_effect: str) -> None:**
   - Updates user-defined lighting preferences based on the location.
3. **update_temperature_preferences(self, location: str, period: str, temperature: float) -> None:**
   - Updates user-set temperature preferences for different periods and locations.
4. **update_fan_speed_preferences(self, location: str, speed: int) -> None:**
   - Updates user-set preferences for fan speed at a specific location.

3.2.2 Appliance Control

1. **control_lights(self, location: str, state: bool, color: str) -> None:**
   - Controls the state and color of lights at a specific location based on user input.
2. **control_thermostats(self, location: str, state: bool, temperature: float) -> None:**
   - Controls the state and temperature settings of thermostats at a specific location based on user input.
3. **control_fans(self, location: str, speed: int) -> None:**
   - Controls the speed of fans at a specific location based on user input.
4. **control_tvs(self, location: str, state: bool, channel: str, volume: int) -> None:**
   - Controls the state, channel, and volume of TVs at a specific location based on user input.

3.2.3 Decision-Making Interface

1. **run_home_automation(self) -> None:**
   - Initiates the decision-making process, interacting with the PPO class to determine optimal actions for each appliance and location.
2. **detect_manual_override(self, location: str) -> bool:**
   - Detects manual overrides by users at a specific location and returns a boolean flag.
3. **update_reward_function(self, location: str, manual_override: bool) -> None:**

- Updates the reward function for the reinforcement learning model based on manual override detection at a specific location.

4. Non-functional Requirements

4.1 Performance

1. **Objective:**
   - Achieve low-latency execution of functions for real-time responsiveness.
2. **Specifications:**
   - Optimize algorithms for efficient variable management and appliance control.
   - Implement asynchronous processing for parallelization.

4.2 Scalability

1. **Objective:**
   - Design the system to accommodate additional appliances, locations, and variables.
2. **Specifications:**
   - Implement a scalable architecture to support the inclusion of new devices, locations, and user preferences.
   - Utilize distributed computing for scalability.

4.3 Security

1. **Objective:**
   - Ensure secure communication with appliances and protect user data.
2. **Specifications:**
   - Implement end-to-end encryption for communication with appliances.
   - Employ secure authentication and authorization mechanisms.

5. System Constraints

1. The system assumes reliable communication with appliances and sensors.
2. Learning patterns may take time and require sufficient historical data for effective decision-making.

6. Dependencies

1. **Hardware:**
   - Reliable and compatible hardware for appliance control, environmental sensing, and location detection.
2. **Software:**

- Deep learning frameworks for interacting with the PPO class.
- Integration with real-time databases for efficient data retrieval.

7. System Interfaces

1. **Input:**
   - User preferences, environmental conditions, manual override signals, and location information.
2. **Output:**
   - Executed actions for each appliance, updated variables, and reward function adjustments.

8. Future Enhancements

1. Integration with additional smart devices and sensors for comprehensive home automation.
2. Implementation of advanced reinforcement learning algorithms for more intelligent decision-making.

9. Glossary

- **PPO:** Proximal Policy Optimization, a reinforcement learning algorithm.

10. Revision History

Version	Date	Author	Description
1.0	2024-02-12	Om Baval	Initial version of the SRS document.
2.0	2024-02-28	Om Baval	Functional Requirements Enhanced

Note:
This document is subject to updates as the project progresses. Adjustments may be made based on feedback, further analysis, and evolving project requirements.